UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación Paralela y Distribuida

Facultad de Ingeniería
Cátedratico: Santiago Padilla



Proyecto 2 - Parte 2: HPC con OpenMPI colectivo

Integrantes:
Jose López – 23773
Fernando Ruiz – 23065
Olivier Viau – 23544

Guatemala, 23 de octubre de 2026

Este informe cubre la Parte 2 de la consigna. La Parte 1 está documentada en el `README.md` del repositorio y en la carpeta `resultados/`.
Repositorio: `https://github.com/JPLopez23/P2-Paralela`

1 Fase de validación: descifrado cruzado local

El equipo definió los dos cruces que pedía esta fase: el equipo intentó descifrar el texto del Grupo 7 ("Gatos explosivos"), y el equipo que intentó descifrar el texto que nuestro equipo publicó en el foro fue también el Grupo 7.

Equipo que eligieron	Texto	Palabra objetivo
Grupo 7 ("Gatos explosivos")	`datos/cifrado_grupo7.des`	tolerancia a fallos

Llave encontrada y verificada: llave numérica (lógica) 12989899, llave DES en hexadecimal `97d6310d01010101`, modo DES-ECB.

La encontró el worker con 5 procesos MPI. Se verificó de forma independiente con la CLI de OpenSSL: al descifrar con esa llave se recuperó exactamente el texto, y al volver a cifrarlo con la misma llave se obtuvo un archivo idéntico (byte a byte, mismo SHA-256) al `cifrado_foro-1.des` original.

El rango que publicó Grupo 7 en el foro (2^{22}) resultó ser incorrecto: la llave real (12989899) lo superaba, pero sí estaba dentro de 2^{24}. El equipo tuvo que ampliar el rango de búsqueda y probar una variante adicional de conversión de la llave lógica a los 8 bytes de DES (7 bits por grupo, en orden little-endian) para encontrarla. El detalle completo de las combinaciones probadas antes de dar con la correcta quedó registrado en `BITACORA.md`.

Mientras tanto, el Grupo 7 también atacó el texto que nuestro equipo publicó en el foro (frase objetivo `escalabilidad horizontal`) y logró descifrarlo correctamente, completando así el cruce en ambos sentidos.

El binario soportó este caso sin modificar el texto original de nadie: en vez de cifrar un texto propio con una llave conocida (modo de autoprueba), leía directamente el archivo cifrado de otro equipo y buscaba la llave que revelara la frase objetivo. Como no había un estándar común entre equipos para convertir la llave lógica de 56 bits a los 8 bytes de DES, el programa admitió varias variantes seleccionables al compilar con `DES_KEY_LAYOUT`; para el texto del Grupo 7 la que funcionó fue la variante 1 (grupos de 7 bits en orden little-endian):

```
make CPPFLAGS=-DDES_KEY_LAYOUT=1
./build/bruteforce_mpi --cipher-file datos/cifrado_grupo7.des --phrase "tolerancia a
  fallos" --upper 16777216
```

Para una corrida más rápida en una sola máquina (local, antes del clúster), se puede paralelizar igual que en la Fase 1:

```
mpirun -np 4 ./build/bruteforce_mpi --cipher-file datos/cifrado_grupo7.des --phrase "
  tolerancia a fallos" --upper 16777216
```

Log de tiempo y speedup, referencia T_s. Se midió con el binario real del proyecto, compilado con `DES_KEY_LAYOUT=1`, buscando en todo el rango 0 a 2^{24} contra el texto del Grupo 7, con 1, 2 y 4 procesos.

Procesos	Tiempo (s)	Speedup vs. 1 proceso	Eficiencia
1	329.74	1.00	100%
2	111.49	2.96	148%
4	11.87	27.79	695%

Ese tiempo local (T_s, con 1 proceso) sirvió de referencia para calcular el speedup real del clúster distribuido en la sección de Despliegue de red física. Los valores de speedup y eficiencia por encima de lo ideal no reflejaron un error de medición ni una ganancia real de eficiencia: en un problema de búsqueda como este, el tiempo dependía de en qué parte del rango le tocaba buscar a cada proceso. Con 4 procesos, a la llave real (12989899, que caía relativamente avanzada dentro del rango de 2^{24}) le tocó un segmento más chico, por lo que se encontró mucho antes que al repartir el mismo rango entre menos procesos. Fue un efecto conocido en búsquedas paralelas y no una medida confiable de eficiencia real del paralelismo; para un número de muestras más representativo habría que haber promediado varias llaves distintas, como se hizo en la Fase 1 con el script de benchmark.

2 Virtualización del entorno: Docker

El Dockerfile en la raíz construyó una imagen `mpi-cluster-image` con `ubuntu:22.04`, `build-essential`, `openmpi-bin`, `libopenmpi-dev`, `libssl-dev` y `openssh-server`. Además habilitó `PermitRootLogin yes` en `/etc/ssh/sshd_config`, fijó `StrictHostKeyChecking no` en `/etc/ssh/ssh_config`, definió `OMPI_ALLOW_RUN_AS_ROOT=1` y `OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1`, y compiló el binario (`make`) al construir la imagen.

```
make docker-build
```

Tabla de validación, ejecutada y verificada el 21 de octubre de 2026:

Validar	Esperado	Funcionó	Qué se tuvo que modificar o hacer
`docker run -rm mpi-cluster-image mpicc -version`	Debería mostrar gcc (Ubuntu 11.4.0-1ubuntu1∼22.04)	Sí	Nada; mpicc ya apunta a gcc vía update-alternatives al instalar openmpi-bin.
`docker run -rm mpi-cluster-image mpirun -np 2 echo "Proceso MPI ejecutado"`	Debería imprimir el mensaje 2 veces	Sí	Hubo que agregar `-allow-run-as-root` al comando; las variables de entorno solas no bastan.
Crear contenedor de prueba: `docker run -it -rm mpi-cluster-image /bin/bash`	Se ejecuta y entra a la terminal del contenedor	Sí	Nada adicional.
`service ssh start`	Comienza el servicio ssh	Sí	Nada; `mkdir -p /run/sshd` ya se hace en el Dockerfile.
`ssh-keygen` y copiar la llave a `authorized_keys`	Genera una clave y se prueba de forma local	Sí	Nada.
`ssh localhost echo "Conexion SSH sin contrasena exitosa"`	Imprime el mensaje porque logra levantar una conexión SSH libre	Sí	Nada; se automatizó en `scripts/docker/start_node.sh`.

Los scripts `scripts/docker/start_node.sh` (arranca sshd y genera la llave de cada nodo) y `scripts/docker/trust_worker.sh` (copia la llave del maestro a un worker) encapsularon estos pasos para no repetirlos a mano en cada contenedor del clúster físico.

Una prueba adicional con el binario real compilado dentro de la imagen confirmó que el programa corría correctamente dentro del contenedor, en ambos modos:

```
Nodo <hostname>, proceso 0 de 4: rango [0, 16)
Llave encontrada: 23
Tiempo: 0.000102 s
Texto: Texto con la frase objetivo
```

```
Nodo <hostname>, proceso 3 de 4: rango [775000, 800000)
Nodo <hostname>, proceso 3: encontro una candidata en 777777
Llave encontrada: 777777
Tiempo: 0.002500 s
Texto: El equipo rival escondio la palabra secreta aqui.
```

La segunda prueba usó `-cipher-file` sobre un binario cifrado con DES real generado aparte, para validar ese modo sin depender del archivo real de otro equipo.

Esa tabla se validó primero de forma individual, pero el 21 de octubre todo el equipo probó la imagen en conjunto: cada integrante construyó la imagen de Docker en su propia computadora (Windows con Docker Desktop y Mac con OrbStack) y confirmó que compilaba y corría sin errores. Con eso quedó validada la sección completa a nivel de equipo, no solo en una máquina.

3 Despliegue de red física

El equipo terminó conectado por una VPN mesh (Tailscale) en vez de un router físico o un hotspot, porque para la corrida final los 3 integrantes ya no estaban en la misma red física. Tailscale crea una red privada virtual (rango 100.64.0.0/10) donde las 3 máquinas se vieron entre sí exactamente como si estuvieran en el mismo router, sin importar en qué casa o red estuviera cada quien.

Antes de llegar a Tailscale, el equipo primero probó con un hotspot de celular (red 172.20.10.0/28) y ahí se resolvieron tres problemas de red específicos del worker Windows, que no estaban previstos en la consigna original. El detalle completo, con cada comando usado para diagnosticar, quedó registrado en `BITACORA.md`.

Docker Desktop en Windows, a diferencia de Mac o Linux, no expuso la red real de la máquina aunque se usara `-network host` y se activara "Enable Host Networking": solo compartió la red interna de la propia VM de Docker. Por eso hubo que publicar el puerto SSH explícitamente en vez de usar `-network host`.

El firewall de Windows, con el adaptador en perfil Público, bloqueaba por defecto el puerto 22 entrante; se agregó una regla mínima, limitada solo a la subred del equipo.

Como el worker Windows no podía usar `-network host`, tampoco podía usar los puertos TCP aleatorios que OpenMPI normalmente abre entre procesos; se fijó y publicó un rango concreto de puertos para ese nodo (ver `scripts/docker/run_cluster.sh`, variable `FIXED_PORTS=1`).

Al pasar a Tailscale apareció un problema adicional, también resuelto: los contenedores sin `-network host` no tenían ninguna interfaz de red propia dentro del rango 100.64.0.0/10, así que `-mca btl_tcp_if_include` no encontraba nada que usar y MPI no podía comunicar los procesos entre sí. Se resolvió agregando una interfaz de red virtual (dummy) dentro de cada contenedor Windows, con la IP de Tailscale de esa máquina asignada ahí, más el flag de kernel `net.ipv4.ip_nonlocal_bind=1` para permitir que el proceso se enlazara a esa dirección. Con eso, OpenMPI encontró una interfaz válida dentro del rango de Tailscale en cada nodo.

Ejecución del Paso B

```
PS> Get-Content cluster_hosts
100.93.43.1 slots=1 max_slots=1
100.110.1.47 slots=1 max_slots=1
100.119.140.37 slots=1 max_slots=1

docker exec mpi_node bash -lc "cd /work && mpirun --allow-run-as-root --hostfile
  cluster_hosts --map-by ppr:1:node --mca oob_tcp_if_include tsmpi --mca
  oob_tcp_static_ipv4_ports 50009 --mca btl self,tcp --mca btl_tcp_if_include tsmpi --
  mca btl_tcp_port_min_v4 50010 --mca btl_tcp_port_range_v4 1 -np 3 ./build/
  bruteforce_mpi --cipher-file datos/cifrado_grupo7.des --phrase 'tolerancia a fallos'
   --upper 16777216"

Nodo 0d9ce0b91b4e, proceso 0 de 3: rango [0, 5592406)
Nodo 39e08d63be0e, proceso 1 de 3: rango [5592406, 11184811)
Nodo 72035ee6bfda, proceso 2 de 3: rango [11184811, 16777216)
Nodo 72035ee6bfda, proceso 2: encontro una candidata en 12989899
Llave encontrada: 12989899
Tiempo: 66.56248 s
Texto: Un cluster de computadoras es un conjunto de maquinas independientes...
Exit code: 0
```

Tres nodos distintos (`0d9ce0b91b4e`, `39e08d63be0e`, `72035ee6bfda` – maestro, un worker, y el worker Windows de este informe, respectivamente) participaron en un solo `mpirun`, y el programa encontró la llave y recuperó el texto correctamente. El nodo `72035ee6bfda` (el worker de este informe) fue el que encontró la candidata.

Paso A: sincronización de llaves SSH inter-nodo. En cada contenedor, maestro y workers, levantado con `-network host` (o con el truco de interfaz dummy en Windows), se corrió:

```
bash scripts/docker/start_node.sh
```

Eso arranca sshd, genera la llave del nodo si no existe, la agrega a su propio `authorized_keys` e imprime su IP y su llave pública. Desde el nodo maestro, se confió en cada worker copiando la llave del maestro a su `authorized_keys`:

```
bash scripts/docker/trust_worker.sh root@<ip_del_worker>
```

La contraseña de root del worker se pidió una sola vez, para la copia; después las conexiones quedaron libres.

Paso B: ejecución del ataque distribuido. Se copió `cluster_hosts.example` a `cluster_hosts` y se reemplazaron las IPs por las de Tailscale de cada integrante, con `slots` y `max_slots` según los núcleos (vCPUs asignadas a Docker) de cada máquina. La imagen se reconstruyó con `DES_KEY_LAYOUT=1` para que todos los nodos usaran la conversión de llave correcta para el texto del Grupo 7. Desde el nodo maestro, con la interfaz virtual `tsmpi` ya creada en cada nodo, se ejecutó:

```
mpirun --allow-run-as-root --hostfile cluster_hosts --map-by ppr:1:node --mca
  oob_tcp_if_include tsmpi --mca oob_tcp_static_ipv4_ports 50009 --mca btl self,tcp --
  mca btl_tcp_if_include tsmpi --mca btl_tcp_port_min_v4 50010 --mca
  btl_tcp_port_range_v4 1 -np 3 ./build/bruteforce_mpi --cipher-file datos/
  cifrado_grupo7.des --phrase "tolerancia a fallos" --upper 16777216
```

El 23 de octubre se capturó la evidencia requerida (ver la salida completa arriba): el comando `mpirun` ejecutado con el `hostfile` visible, la salida mostrando los distintos nodos participando, y el mensaje final con la llave encontrada (12989899) y el texto recuperado.

4 Discusión de resultados y conclusiones

Por qué OpenMPI se congela indefinidamente sin `-mca btl_tcp_if_include`: cada contenedor Docker tiene, además de la interfaz física, una interfaz virtual de bridge, `docker0`, típicamente en 172.17.0.0/16. Al arrancar, el motor de transporte TCP de OpenMPI (BTL) y el módulo de out-of-band (OOB, usado para el handshake inicial entre demonios `orted`) enumeran todas las interfaces de red disponibles y anuncian sus IPs a los demás procesos como posibles rutas. Sin restricción, un nodo le puede anunciar a otro su IP de `docker0` como si fuera alcanzable. El proceso remoto intenta conectarse a esa IP, pero esa dirección solo existe dentro de la red virtual local de cada máquina física, no es enrutable a través del router. El SYN del connect nunca llega a destino ni recibe un RST, así que el socket se queda esperando en el timeout por defecto del kernel, que es largo. Como MPI necesita que todos los pares completen su conexión antes de continuar, el programa completo parece colgado sin ningún mensaje de error. Los parámetros `btl_tcp_if_include` y `oob_tcp_if_include` con la subred física fuerzan a OpenMPI a anunciar y usar únicamente esa subred real, eliminando la ambigüedad.

Problemas de red enfrentados: el firewall de Windows (perfil Público) bloqueó por defecto el

puerto SSH y cualquier puerto TCP sin regla explícita, así que hubo que crear reglas de entrada limitadas a la subred del equipo (primero el hotspot, luego la de Tailscale) para el puerto 22 y el rango de MPI. Docker Desktop en Windows no compartió la red real de la máquina con `-network host`, a diferencia de Mac o Linux, lo que obligó a publicar puertos explícitos y luego, para que OpenMPI encontrara una interfaz válida dentro del rango de Tailscale, a crear una interfaz dummy dentro del contenedor con la IP de Tailscale asignada. El equipo dejó de poder estar en la misma red física a mitad del proyecto, lo que obligó a migrar del hotspot a una VPN mesh (Tailscale). También hubo problemas de cuenta en Tailscale (un integrante quedó en su propia tailnet personal por accidente en vez de la del equipo, y tuvo que cerrar sesión y volver a entrar desde el correo de invitación), y un cambio de rol sobre la marcha porque el nodo maestro planeado originalmente no logró completar la conexión a tiempo. Además, el rango de llave que publicó el Grupo 7 en el foro resultó ser incorrecto (indicaron 2^{22} cuando la llave real necesitaba 2^{24}), lo que retrasó bastante encontrar su llave.

Tiempo local contra clúster físico: el tiempo local (T_s, una sola máquina, 1 proceso) fue 329.74 segundos. El tiempo del clúster real, con Tailscale y 3 máquinas físicas distintas corriendo 3 procesos, fue 66.56 segundos. Speedup real: $329.74/66.56 \approx 4.95$ veces, con solo 3 procesos en 3 máquinas distintas. Igual que con las mediciones locales, este número es superlineal respecto al speedup ideal (que para 3 procesos sería como mucho 3 veces), no porque el clúster sea más eficiente que la teoría, sino porque la llave real cayó en el tercer segmento del rango, el que le tocó justo al nodo que la encontró, así que ese proceso no tuvo que recorrer tanto como le hubiera tocado a un solo proceso buscando todo el rango. Aun así es una señal positiva: la latencia de red (VPN más SSH más sincronización por lote) no impidió que el clúster terminara varias veces más rápido que la corrida local, a pesar del overhead adicional de coordinar 3 máquinas distintas por una red virtual.

Qué pasa con el rendimiento si se configuran 32 procesos MPI en una computadora con 4 núcleos físicos: el sistema operativo igual los deja correr, pero el rendimiento se degrada: los 32 procesos compiten por solo 4 núcleos lógicos, forzando al planificador del sistema operativo a hacer cambios de contexto constantes entre ellos. Para un algoritmo como la fuerza bruta DES, que es intensivo en CPU sin esperas de entrada o salida, cada cambio de contexto es puro overhead, sin avanzar trabajo útil. Además, las sincronizaciones por lote obligan a esperar a los 32 procesos en cada ronda, y el proceso más lento frena a los demás. El resultado típico es que el tiempo total termina siendo peor que con 4 procesos, no mejor, de ahí la recomendación de declarar `max_slots` igual a los núcleos físicos reales.

Vulnerabilidades y retos logísticos de HPC con hardware heterogéneo: permitir el acceso root por SSH y desactivar la verificación de huellas de host son aceptables para un ejercicio de clase en una LAN controlada, pero en un clúster real son vulnerabilidades serias: cualquiera en la red con acceso a un nodo comprometido puede saltar a los demás como root sin fricción. Una vez que las llaves SSH están sincronizadas entre todos los nodos, comprometer un solo contenedor equivale a comprometer el clúster completo. La heterogeneidad de hardware, núcleos, velocidad de CPU y memoria distintos entre las máquinas de cada integrante, hace que un reparto parejo del rango de llaves deje a los nodos más lentos como cuello de botella, porque los más rápidos esperan en la siguiente sincronización. La dependencia de la red física, firewalls, NAT de Docker, cambios de IP por DHCP y distinta configuración de cada sistema operativo, multiplica los puntos de falla fuera del control del código. Administrar varios nodos a mano, como en este laboratorio, no escala; en producción se necesitan herramientas de orquestación para automatizar lo que aquí se hizo manualmente con scripts.

Conclusión del equipo: el equipo logró el objetivo de la segunda parte del proyecto. La parte que más le gustó al equipo fue entender cómo coordinar Docker entre los tres integrantes para formar un clúster real, y sobre todo aplicar el cifrado y descifrado DES para revelar las llaves de los

textos de otros grupos: ver ese proceso funcionando en la práctica, más allá de la teoría, resultó interesante para el equipo. La parte que menos le gustó fue la coordinación de red al inicio: la conexión física entre las máquinas falló varias veces y obligó a conectarse de forma remota mediante una VPN (Tailscale) en vez de un router o un hotspot compartido, lo que tomó más tiempo del esperado. De cara a una próxima vez, el equipo consideró que pudo haber organizado mejor el orden de trabajo desde el principio, en vez de resolver los problemas de red sobre la marcha conforme fueron apareciendo.